

Projet de Compilation 2025-2026

L3 Informatique

Léo Pierrat

Mai 2026

Manuel d'utilisation

Le compilateur se construit avec :

`make`

L'exécutable produit est `bin/tpcc`. Il s'utilise de la façon suivante :

`./bin/tpcc [options] < monfichier.tpc`

Options disponibles :

Option	Description
<code>-t, --tree</code>	Affiche l'arbre abstrait sur la sortie standard
<code>-s, --syntabs</code>	Affiche toutes les tables des symboles
<code>-h, --help</code>	Affiche l'aide et termine

Codes de retour :

Code	Signification
0	Programme valide (même avec des avertissements)
1	Erreur lexicale ou syntaxique
2	Erreur sémantique

Description du compilateur

Analyse lexicale et syntaxique

L'analyse lexicale est réalisée avec Flex. Le lexeur gère les commentaires multi-lignes `/* ... */` via un état exclusif `COMM`, ce qui permet de continuer à compter les numéros de ligne à l'intérieur des commentaires. Les commentaires mono-ligne `// ...` sont ignorés par une règle simple. Les types `int` et `char` sont renvoyés comme token `TYPE` avec leur valeur, ce qui permet au parseur de les traiter uniformément.

L'analyse syntaxique est réalisée avec Bison. La grammaire implémente le langage TPC complet, y compris les types structure `struct X { ... }`. À chaque règle de grammaire correspond une action

qui construit un nœud de l'arbre abstrait via le module `tree`. Chaque nœud contient un label (type du nœud) et selon le cas un entier, un caractère ou une chaîne (identifiant ou opérateur).

Cette partie a été faite entièrement au premier semestre.

Analyse sémantique

L'analyse sémantique est réalisée dans le module `sem.c`, par un parcours en profondeur de l'arbre abstrait. Le module contient une fonction `sem()`, qui fait un parcours en profondeur récursif, et adapte son comportement selon le nœud. Chaque nœud possède un champ `label`, construit par la grammaire, et permet d'identifier les statuts de chaque nœud. Un `switch case` couvrant tous les types de nœuds est présent dans cette fonction.

Tables de symboles

Quatre tables de hachage sont utilisées (module `symb.c`) :

- `global_table` : variables globales
- `local_table` : variables locales à la fonction en cours (réinitialisée à chaque nouvelle fonction)
- `function_table` : noms et signatures des fonctions déclarées
- `struct_table` : noms des types struct déclarés

Chaque entrée de `function_table` encode, dans le champ `deplct`, le nombre de paramètres et le caractère void du type de retour : `deplct = param_count * 2 + is_void`. Cela permet de vérifier le nombre d'arguments et d'identifier les fonctions void sans structure de données supplémentaire.

Pré-passe

Avant le parcours principal, une pré-passe sur le nœud racine collecte tous les noms de fonctions. Cela permet de gérer les appels mutuellement récursifs et les appels à des fonctions déclarées plus loin dans le fichier source, sans fausse erreur. Les quatre fonctions built-in `putchar`, `getchar`, `putint`, `getint` sont également insérées dans `function_table` au démarrage.

Suivi des types struct (`struct_info`)

Le module `struct_info.c` maintient deux structures :

- `struct_defs` : liste chaînée de `StructDef`, chacune contenant la liste de ses champs (`FieldDef`) avec leur nom, leur indice et leur type (primitif ou struct imbriquée). La présence du type de champ (`struct_type`) permet de résoudre récursivement le type d'un accès enchaîné comme `b.a.x`.
- `var_types` : liste chaînée associant chaque variable de type struct à son nom de type. Cette liste est utilisée pour déterminer à l'analyse si un identifiant désigne une variable struct.

Pour éviter la pollution de portée entre fonctions (une variable `p` déclarée dans `f` ne doit pas être reconnue comme struct dans `g`), le pointeur de tête de `var_types` est sauvegardé au début de chaque fonction et restauré à la fin.

La fonction `get_node_struct_type(node)` infère récursivement le type struct d'une expression : elle retourne le nom du type si le nœud est un identifiant struct ou un accès de champ struct, et `NULL` sinon. Cette fonction est utilisée dans les vérifications de retour et d'affectation.

Erreurs sémantiques détectées

Erreurs sur la fonction `main` :

- `main` absente du programme
- `main` ne retourne pas `int` (retour `void`, `char` ou `struct`)
- `main` déclarée avec des paramètres

Déclarations :

- Variable utilisée sans être déclarée (locale ou globale)
- Fonction appelée sans être déclarée
- Type `struct X` utilisé alors que `X` n'a pas été déclaré (en variable, paramètre ou type de retour)
- Redéclaration d'une variable (locale ou globale)
- Redéclaration d'une fonction
- Redéclaration d'un type `struct`
- Redéclaration d'un champ dans un `struct`
- Redéclaration d'un paramètre
- Conflit entre une variable globale et une fonction de même nom

Appels de fonctions :

- Nombre d'arguments incorrect (trop peu ou trop)
- Résultat d'une fonction `void` utilisé dans une expression (affectation, condition, arithmétique, argument d'appel, instruction `return`)

Instructions `return` :

- `return expr` dans une fonction `void`
- `return` d'une valeur `struct` dans une fonction retournant un type primitif
- `return` d'une valeur non-`struct` dans une fonction retournant un `struct`

Opérations sur les `structs` :

- `Struct` utilisée comme opérande d'une opération arithmétique (+, -, *, /)
- `Struct` utilisée dans une comparaison (==, !=, <, >, <=, >=)
- `Struct` utilisée dans une opération logique (&&, ||, !)
- `Struct` utilisée comme condition d'un `if` ou d'un `while`
- Accès à un champ inexistant d'un `struct`
- Accès de champ sur une variable qui n'est pas de type `struct`
- Accès de champ enchaîné sur un champ de type primitif (`p.x.y` quand `x` est `int`)
- Accès de champ enchaîné invalide sur un champ `struct` (`b.a.z` quand `z` n'existe pas dans le type de `a`)

Affectations impliquant des `structs` :

- Affectation d'une valeur non-`struct` à une variable de type `struct`
- Affectation d'une variable `struct` à une variable de type primitif
- Affectation entre deux variables `struct` de types incompatibles
- Affectation d'un champ de type primitif à une variable `struct` (`p = q.x` quand `x` est `int`)
- Affectation d'un champ de type `struct` à une variable de type primitif (`n = b.a` quand `a` est un `struct`)

Avertissements

- Affectation d'une expression de type `int` à une variable de type `char`

Génération de code

Le code cible est de l'assembleur NASM x86-64 avec les conventions d'appel AMD64. Le fichier de sortie est `_anonymous.asm` par défaut (lecture sur l'entrée standard). En cas d'erreur sémantique, ce fichier est supprimé et le code de retour est 2.

Structure du programme généré. Le fichier assembleur contient :

- Un point d'entrée `_start` qui appelle `main` puis effectue le syscall `exit` avec le code de retour
- Les quatre fonctions built-in d'I/O écrites directement en assembleur
- Les fonctions du programme source
- Une section `.bss` pour les variables globales

Expressions. Les expressions sont évaluées par une approche à pile : chaque sous-expression empile son résultat, les opérateurs dépilent leurs opérandes dans `rax` et `r10` et empilent le résultat. La division utilise `idiv` après `xor rdx, rdx`. La multiplication utilise la forme à deux opérandes `imul rax, r10`.

Booléens. Les expressions booléennes (`&&`, `||`, `!`, `==`, `!=`, `<`, `>`, `<=`, `>=`) utilisent l'évaluation paresseuse (court-circuit) avec des labels. Chaque nœud booléen reçoit un `label_true` et un `label_false` et y saute directement sans passer par la pile.

Fonctions. Chaque fonction génère un prologue (`push rbp ; mov rbp, rsp ; sub rsp, N`) et un épilogue (`mov rsp, rbp ; pop rbp ; ret`). Les arguments sont passés dans les registres `rdi`, `rsi`, `rdx`, `rcx`, `r8`, `r9` et copiés sur la pile locale dans le prologue. Les variables locales sont allouées avec des déplacements négatifs par rapport à `rbp`.

Variables globales. Les variables globales sont allouées en section `.bss` via `resq`. Pour les variables de type struct, le nombre de quadwords réservés correspond au nombre de champs du struct. Les accès en lecture et écriture se font par label (`mov rax, [nom]` et `mov [nom], rax`).

Structs. Les champs d'un struct sont disposés consécutivement sur la pile (ou en `.bss`), séparés de 8 octets. L'indice de champ (calculé lors de l'enregistrement du struct) donne directement le décalage : `[rbp - offset - idx * 8]` pour une variable locale, `[label + idx * 8]` pour une variable globale.

Fonctions I/O built-in. Les quatre fonctions sont écrites directement en NASM en utilisant uniquement les syscalls (`rax=0` pour `read`, `rax=1` pour `write`, `rax=60` pour `exit`). `putint` convertit l'entier en chaîne de caractères manuellement par divisions successives.

Difficultés rencontrées

Tables des symboles. La première grosse difficulté que j'ai rencontré est l'implémentation primaire des tables des symboles. J'ai eu du mal à comprendre ce que je devais faire pour commencer à construire, donc principalement le début du module `sem.c` et les profondeurs du module `symb.c`. Mais une fois que j'ai eu le plan clair de ce que je voulais faire, c'était moins dur et ça s'est déroulé.

Contrôle de types. J'ai eu aussi du mal à comprendre ce qu'on attendait concernant le contrôle de types, mais finalement ce n'était pas très dur. Au début je pensais faire un système de structures

contenant les arbres de types, ceux du cours, mais finalement j'ai fait une fonction qui prend un noeud et en déduit son type. Cette façon de faire est je pense trop simpliste pour du C, mais elle ne pose pas problème dans le contexte du tpc, puisqu'on a pas de pointeurs et de types complexes (sauf pour les structs, j'y viens). Pour les structs, la méthode est la même, une fonction qui trouve le type struct d'un noeud, mais elle est un peu plus complexe.

Bancs de tests. Je n'ai pas réussi à fixer tous les bancs de tests, et le problème c'est qu'il faudrait vraiment que j'aie un recul énorme sur le projet pour trouver les failles. Je pense que si c'était de vie ou de mort, la méthode aurait été de faire un "schéma" du projet, de son architecture, mais c'est un travail qui aurait été mieux avant de commencer à coder. C'est typiquement le genre de choses que j'aurais trouvé horriblement dur à faire sans l'IA (un paragraphe dédié sur l'ia sera présent après)

Implémentation des structs. Je pense que l'implémentation des structs était une difficulté, parce que je trouve que déjà sans l'extension du projet, la base (core) est déjà ardue, mais les structs m'ont donné l'impression de devoir refaire tout de façon différente pour le cas spécial des structs, que ce soit les tables des symboles, le contrôle des types, les erreurs d'évaluation, d'assignation, etc. De plus,

Gestion du temps J'ai trouvé vraiment dur la façon dont était organisé le semestre par rapport au projet, le projet est sûrement le projet le plus long qu'on ait fait, mais c'était difficile de rester avec les tp, parce que par exemple, étant donné que j'ai bloqué sur le tp1, alors j'ai passé beaucoup de temps à ne pas faire les tps au jour le jour parce que j'avais eu un blocage dès le début. À la fin du semestre, ça allait mieux, mais la période de fin était trop intense je pense, parce que je n'ai pas réussi à faire en sorte d'avoir beaucoup de temps pour le projet comparé aux examens. Après, je ne suis pas inquiet quant au semestre, mais je pense que j'ai mal jaugé le temps que ce projet prend. Je l'ai trouvé beaucoup plus long que par exemple le projet d'analyse syntaxique ou le projet de codec différentiel. Et comme je trouve que les modules ce semestre sont infiniment plus durs que ceux du premier semestre, alors c'était compliqué de m'investir sur le projet en priorité.

Solo Je pense aussi que le fait que je sois tout seul m'a causé beaucoup de problèmes. Au projet d'analyse syntaxique, ça ne m'a pas posé trop de problèmes, mais sur ce projet, qui est beaucoup plus ambitieux, j'aurais aimé avoir de l'aide. Le fait que je sois tout seul est principalement hérité du fait que j'étais tout seul en analyse syntaxique, parce que dans mon groupe de tp du premier semestre je ne connaissais personne. Et je ne savais pas qu'on pouvait se mettre avec quelqu'un d'un autre groupe. Si je le savais, je pense que je me serais mis avec mon ami Samir, qui lui aussi est tout seul.

IA

Utilisation J'ai utilisé gpt et claude code au travers du projet, même si mon utilisation de claude code était assez minime (surtout pour de l'édition/formatage), et n'est représentée que sur les derniers jours. Mais sinon j'ai utilisé surtout gpt avec la fonction de lecture des repos github. J'ai surtout utilisé l'ia pour m'aiguiller dans mes difficultés. Typiquement, au début du projet, quand je ne comprenais pas ce qu'on attendait pour le parcours de l'arbre abstrait, c'est gpt qui m'a suggéré une structure de grosse fonction récursive avec des switch case. Parfois je n'ai pas eu besoin de l'ia, comme pour les problèmes que j'avais eu pour le contrôle de types, mais en général j'ai fait avec. J'ai aussi utilisé l'ia pour me générer la fonction d'affichage des tables des symboles, et le script de test `run_tests`. Enfin comme j'ai dit plus tôt, j'ai utilisé l'ia pour chercher les erreurs sémantiques/d'exécution.

Divers L'ia m'a suggéré également l'utilisation de l'outil pandoc pour construire le rapport avec le .md, et m'a construit une ébauche (même si j'écris le rapport à la main).